

Benefits Service Consumer Guide

Consumer-facing guide for applications that need to call Benefits Service.
Last updated: 2026-07-06

Overview

Benefits Service provides customer-scoped plan benefit data and coverage-check decisions. Consumers can:

- list loaded benefit plans
- read the latest or effective-date-specific payload for one plan
- run a coverage check for a resolved plan/product and one or more code pairs
- upload plan snapshots, when explicitly approved for ingestion

All deployed consumer calls go through APIM and authenticate with an `x-api-key` header. Consumers do not pass a customer slug in the request and do not create or send `x-context-token` values. APIM validates the API key with Customer Configuration Service (CCS), receives an internal context token, and forwards the request to Benefits Service with trusted customer context.

Integration Configuration

Use the URL format and environment-specific API Management route paths in this guide to configure the Benefits Service URL. Obtain the customer-specific access details for your integration from Stellarus.

Value	Source	Description
<code>BENEFITS_BASE_URL</code>	This guide	Shared API base URL: <code>https://api.stellarus.com</code> .
<code>BENEFITS_ENVIRONMENT</code>	This guide / integration target	Target environment: <code>dev</code> , <code>qa</code> , <code>stg</code> , or <code>prd</code> .
<code>BENEFITS_APIM_ROUTE_PATH</code>	This guide	Env-specific API Management route/path: <code>/benefits-service-{environment}</code> .
<code>BENEFITS_API_KEY</code>	Stellarus	Customer-specific API key issued by CCS and delivered once through a secure channel.
Allowed endpoint operations	Stellarus	Benefits method/resource-path operations your system is approved to call.
Approved scopes	Stellarus	Usually <code>benefits:read</code> for reads/coverage checks or <code>benefits:write</code> for uploads.

The complete URL for Benefits Service will look like:

```
https://api.stellarus.com/benefits-service-{environment}/{resource-path}
```

Allowed `{environment}` values: `dev` | `qa` | `stg` | `prd`.

Environment	API Management route/path	Composed service URL
dev	/benefits-service-dev	https://api.stellarus.com/benefits-service-dev
qa	/benefits-service-qa	https://api.stellarus.com/benefits-service-qa
stg	/benefits-service-stg	https://api.stellarus.com/benefits-service-stg
prd	/benefits-service-prd	https://api.stellarus.com/benefits-service-prd

Endpoint example template for Get Single Plan:

```
GET https://api.stellarus.com/benefits-service-{environment}/api/v1/plans/{planId}
GET https://api.stellarus.com/benefits-service-{environment}/api/v1/plans/{planId}?effectiveDate={YYYYMMDD}
```

`effectiveDate` is optional. Omit it to retrieve the latest loaded effective date for the plan. Include it when you need the plan payload for a specific effective date, such as a specific plan year.

Expanded examples:

Environment	Get Single Plan endpoint
dev	GET https://api.stellarus.com/benefits-service-dev/api/v1/plans/{planId}
qa	GET https://api.stellarus.com/benefits-service-qa/api/v1/plans/{planId}
stg	GET https://api.stellarus.com/benefits-service-stg/api/v1/plans/{planId}
prd	GET https://api.stellarus.com/benefits-service-prd/api/v1/plans/{planId}

Specific effective-date example:

```
GET https://api.stellarus.com/benefits-service-stg/api/v1/plans/M0005767?effectiveDate=20260101
```

The examples below assume:

```
export BENEFITS_BASE_URL="https://api.stellarus.com"
export BENEFITS_ENVIRONMENT="stg"
export BENEFITS_APIM_ROUTE_PATH="/benefits-service-${BENEFITS_ENVIRONMENT}"
export BENEFITS_SERVICE_URL="${BENEFITS_BASE_URL}${BENEFITS_APIM_ROUTE_PATH}"
export BENEFITS_API_KEY="<your-issued-api-key>"
```

Authentication

Send the API key on every request:

```
x-api-key: sk_test_or_live_...
```

For JSON POST requests, also send:

```
content-type: application/json
```

Do not send any customer specific information in the header. For an incoming request to Benefits Service, APIM strips inbound `x-*` headers, validates `x-api-key` with CCS, and injects the internal headers that Benefits Service verifies.

Rate Limits And Concurrency

Benefits Service is protected at the APIM/API-key layer. The default API rate limit is **60 requests per second per API key**, unless Stellarus has configured a customer-specific override.

If your system exceeds its assigned limit, APIM returns `429 Too Many Requests` with a `Retry-After` header. Consumers should honor `Retry-After`, keep aggregate traffic below the assigned RPS limit, and use bounded client-side concurrency with retry backoff and jitter.

There is no separate published concurrency limit for Benefits Service. If your expected production traffic requires more than the default 60 RPS, coordinate with Stellarus before launch.

Requesting An API Key

New consumers cannot self-issue API keys. Request one from the Stellarus platform or Benefits Service owner.

Include this information in the request:

Field	Required	Example
Customer name and slug	Yes	<code>Acme Health, acme</code>
Environment	Yes	<code>dev, qa, stg, prd</code>
Consumer system	Yes	<code>prior-auth-api, claims-adjudication-batch</code>
Business owner	Yes	Team or person accountable for the use case
Technical owner	Yes	Team or person responsible for integration and rotation
Use case	Yes	<code>coverage check, plan read, plan upload</code>
Endpoint operations needed	Yes	<code>GET /api/v1/plans/{planId}</code>
Scopes requested	Yes	<code>benefits:read</code> OR <code>benefits:write</code>
Mode	Yes	<code>test</code> OR <code>live</code>
Expiration	Recommended	<code>2026-12-31T23:59:59Z</code>
Delivery channel	Yes	Preferred secure channel

Suggested request template:

Request an API key for Benefits Service.

```
Customer:
Environment:
Consumer system:
Business owner:
Technical owner:
Use case:
Endpoint operations:
Scopes:
Mode:
Expires at:
Delivery channel:
```

Stellarus will issue the key through CCS, normally via `customer-admin-ui`. The plaintext key is shown exactly once during creation. It is not stored by Stellarus and cannot be retrieved later. If the key is lost, it must be revoked and replaced.

Scopes

Request the smallest scope set needed for your use case.

Scope	Allows	Endpoint operations
<code>benefits:read</code>	Plan metadata reads, plan payload reads, coverage checks	<code>GET /api/v1/plans</code> , <code>GET /api/v1/plans/{planId}</code> , <code>POST /api/v1/coverage-check</code>
<code>benefits:write</code>	Plan snapshot uploads	<code>POST /api/v1/plans</code>

`benefits:write` is restricted to approved ingestion clients. Read-only consumers should not request it.

Resource Path Summary

Combine the HTTP method and resource path below with `BENEFITS_SERVICE_URL` to form the full endpoint.

Method	Resource path	Purpose	Scope
<code>GET</code>	<code>/api/v1/plans</code>	List loaded plan metadata for the resolved customer.	<code>benefits:read</code>
<code>GET</code>	<code>/api/v1/plans/{planId}</code>	Read one plan payload. Returns latest effective date unless <code>effectiveDate</code> is supplied.	<code>benefits:read</code>
<code>POST</code>	<code>/api/v1/coverage-check</code>	Evaluate coverage for a caller-resolved plan/product and one or more procedure/diagnosis pairs.	<code>benefits:read</code>
<code>POST</code>	<code>/api/v1/plans</code>	Upload one or more plan snapshots.	<code>benefits:write</code>

List Plan Metadata

Returns loaded plan metadata for the customer resolved from your API key.

```
GET /api/v1/plans
```

Query parameters:

Parameter	Type	Required	Description
<code>limit</code>	Integer	No	Page size from <code>1</code> to <code>1000</code> . Default is <code>100</code> .
<code>cursor</code>	String	No	Opaque cursor returned as <code>nextCursor</code> from the previous page.

Sample request:

```
curl -sS "${BENEFITS_SERVICE_URL}/api/v1/plans?limit=100" \
-H "x-api-key: ${BENEFITS_API_KEY}"
```

Sample `200 OK` response:

```
{
  "customerId": "11111111-1111-4111-8111-111111111111",
  "customerSlug": "acme",
  "plans": [
    {
      "planId": "M0005767",
      "effectiveDate": "20260101",
      "planName": "Gold Plus PPO",
      "productLob": "Commercial"
    }
  ],
  "count": 1,
  "nextCursor": null,
  "hasMore": false
}
```

Expected errors:

Status	Meaning
<code>400</code>	Invalid <code>limit</code> or malformed <code>cursor</code> .
<code>401</code>	Missing, expired, revoked, or invalid API key.
<code>403</code>	API key does not include <code>benefits:read</code> .
<code>503</code>	Benefits Service or database unavailable.

Read One Plan

Returns the raw stored plan payload for one plan. The `effectiveDate` query parameter is optional:

- Omit `effectiveDate` to return the latest loaded effective date for the plan.
- Include `effectiveDate=YYYYMMDD` to return a specific effective-date version, such as a plan payload for a specific plan year.

```
GET /api/v1/plans/{planId}
GET /api/v1/plans/{planId}?effectiveDate={YYYYMMDD}
```

Path parameters:

Parameter	Type	Required	Description
planId	String	Yes	Plan identifier, for example M0005767 .

Query parameters:

Parameter	Type	Required	Description
effectiveDate	String	No	YYYYMMDD . When omitted, the latest effective date for the plan is returned.

Sample request for the latest effective date:

```
curl -sS "${BENEFITS_SERVICE_URL}/api/v1/plans/M0005767" \
-H "x-api-key: ${BENEFITS_API_KEY}"
```

Sample request for a specific effective date:

```
curl -sS "${BENEFITS_SERVICE_URL}/api/v1/plans/M0005767?effectiveDate=20260101" \
-H "x-api-key: ${BENEFITS_API_KEY}"
```

Sample 200 OK response:

```
{
  "id": "22222222-2222-4222-8222-222222222222",
  "customerId": "11111111-1111-4111-8111-111111111111",
  "customerSlug": "acme",
  "planId": "M0005767",
  "effectiveDate": "20260101",
  "sourceName": "nexus",
  "sourceId": "M0005767-20260101",
  "updatedAt": "2026-07-06T18:30:00.000Z",
  "data": {
    "planName": "Gold Plus PPO",
    "deductible": 1000
  }
}
```

Expected errors:

Status	Meaning
400	Invalid effectiveDate format.
401	Missing, expired, revoked, or invalid API key.
403	API key does not include benefits:read .
404	No matching plan found for the resolved customer and requested effective date.
503	Benefits Service or database unavailable.

Run Coverage Check

Evaluates coverage for one resolved plan/product and one or more procedure/diagnosis/place-of-service triples.

Benefits Service does not perform member lookup or eligibility resolution. The caller must already know the `planId` and `productId`.

POST `/api/v1/coverage-check`

Request body:

Field	Type	Required	Description
<code>planId</code>	String	Yes	Caller-resolved plan identifier. Must end in digits for suffix-tolerant matching.
<code>productId</code>	String	Yes	Caller-resolved product identifier. Must end in digits for suffix-tolerant matching.
<code>codes</code>	Array of objects	Yes	One or more code checks.
<code>codes[].procCode</code>	String	Yes	Procedure, CPT, or HCPCS-style code.
<code>codes[].dxCode</code>	String	Yes	Diagnosis code.
<code>codes[].placeOfService</code>	String	Yes	Place of service. Accepted and audited.
<code>dateOfService</code>	String	No	<code>YYYY-MM-DD</code> . Accepted and audited.

Sample request:

```
curl -sS -X POST "${BENEFITS_SERVICE_URL}/api/v1/coverage-check" \
-H "content-type: application/json" \
-H "x-api-key: ${BENEFITS_API_KEY}" \
-d '{
  "planId": "M0005767",
  "productId": "M0005767",
  "codes": [
    {
      "procCode": "99213",
      "dxCode": "I10",
      "placeOfService": "11"
    }
  ],
  "dateOfService": "2026-01-15"
}'
```

Sample 200 OK response:

```
{
  "input": {
    "planId": "M0005767",
    "productId": "M0005767",
    "dateOfService": "2026-01-15"
  },
  "resolved": {
    "planId": "33333333-3333-4333-8333-333333333333",
    "planName": "Gold Plus PPO",
    "planType": "PPO",
    "matchedPlanCode": "M0005767",
    "matchedProductCode": "M0005767",
    "internalProductId": "44444444-4444-4444-8444-444444444444"
  },
  "codes": [
    {
      "procCode": "99213",
      "dxCode": "I10",
      "placeOfService": "11",
      "covered": true,
      "coverage": [
        {
          "covered": true,
          "procedureDescription": "Office or other outpatient visit",
          "diagnosisDescription": null,
          "category": "OFFICE_VISIT",
          "benefitService": "PRIMARY_CARE",
          "costSharingType": "COPAY",
          "copayAmount": 25,
          "coinsurancePct": null,
          "perUnit": "per_visit",
          "reason": null,
          "priorAuthRequired": null,
          "referralRequired": null,
          "notes": null
        }
      ]
    }
  ]
}
```

Expected errors:

Status	Meaning
400	Invalid request body, identifiers without trailing digits, or ambiguous plan/product match.
401	Missing, expired, revoked, or invalid API key.
403	API key does not include <code>benefits:read</code> .
404	No matching plan/product found for the resolved customer.
503	Benefits Service or database unavailable.

Upload Plan Snapshots

Uploads one or more plan snapshots. This endpoint is for approved ingestion clients only and requires a dedicated key with `benefits:write`.

```
POST /api/v1/plans
```

Request body:

Field	Type	Required	Description
<code>plans</code>	Array of objects	Yes	One or more plan snapshot items.
<code>plans[].plan_id</code>	String	Yes	Plan identifier.
<code>plans[].effective_date</code>	String	Yes	<code>YYYYMMDD</code> . Part of the upsert key.
<code>plans[].payload</code>	Object	Yes	Raw plan document object.
<code>plans[].source_name</code>	String	No	Upstream source label, for example <code>nexus</code> .
<code>plans[].source_id</code>	String	No	Upstream source identifier for this snapshot.
<code>plans[].plan_name</code>	String	No	Plan display name for metadata listing.
<code>plans[].product_lob</code>	String	No	Product line of business for metadata listing.

Sample request:

```
curl -sS -X POST "${BENEFITS_SERVICE_URL}/api/v1/plans" \
-H "content-type: application/json" \
-H "x-api-key: ${BENEFITS_API_KEY}" \
-d '{
  "plans": [
    {
      "plan_id": "M0005767",
      "effective_date": "20260101",
      "source_name": "nexus",
      "source_id": "M0005767-20260101",
      "plan_name": "Gold Plus PPO",
      "product_lob": "Commercial",
      "payload": {
        "planName": "Gold Plus PPO",
        "deductible": 1000
      }
    }
  ]
}'
```

Sample `201 Created` response:

```
{
  "accepted": 1,
  "customerId": "11111111-1111-4111-8111-111111111111",
  "customerSlug": "acme",
  "receivedAt": "2026-07-06T18:30:00.000Z"
}
```

Expected errors:

Status	Meaning
400	Missing or invalid request body, or an item is missing <code>plan_id</code> , <code>effective_date</code> , or <code>payload</code> .
401	Missing, expired, revoked, or invalid API key.
403	API key does not include <code>benefits:write</code> .
413	Request body exceeds the service limit. Split the batch and retry.
503	Benefits Service or database unavailable.

Error Response Shape

Error responses use this general shape. The exact `message` may vary by error condition.

```
{
  "statusCode": 403,
  "message": "Forbidden resource",
  "error": "Forbidden"
}
```

Recommended handling:

Status	Consumer action
400	Fix the request before retrying.
401	Confirm the API key is present, current, and from the right environment. Escalate if it should be valid.
403	Use a key with the required scope or request a scope change.
404	Treat as valid authentication with no matching data for the requested customer/identifier.
413	Split upload batches into smaller requests.
429	Respect <code>Retry-After</code> ; the default APIM/API-key limit is 60 requests per second unless overridden for your customer.
5xx	Retry according to your integration policy and escalate if persistent.

Operational Guidance

- Store the API key in a secret manager.
- Never commit the API key to source control.
- Never log the `x-api-key` header.
- Rotate keys on your agreed cadence and immediately after suspected exposure.
- If a key is lost, request revocation and reissue. Stellarus cannot recover the plaintext value.

Quick Smoke Test

After receiving a read key, this call is usually the lightest authentication check:

```
curl -i "${BENEFITS_SERVICE_URL}/api/v1/plans?limit=1" \
-H "x-api-key: ${BENEFITS_API_KEY}"
```

Interpretation:

Result	Meaning
200	Key is valid and has <code>benefits:read</code> .
401	Key is missing, invalid, expired, revoked, or for the wrong route/environment.
403	Key is valid but lacks <code>benefits:read</code> .
5xx	Platform dependency issue or service unavailability.